ÉPREUVE PRATIQUE — ADRIA PFE 27

Architecture Micro-services, IA et Développement Full-Stack

Candidat : Mohammed Yassine Lamhamdi El Alaoui

Dépôt GitHub : `lamhamdi-el-alaoui-mohammed-yassine-test-adria-pfe-27`

Technologies : Spring Cloud, Spring Boot 3.5, Spring AI, Kafka, Keycloak, Angular/React, Python ML, Docker

Date d'évaluation : 30 Octobre 2026 | Durée : 3 Heures

Sommaire du Rapport :

- Partie A — Préparation, Convention Git & Structure Modulaire
- Partie B — Conception et Architecture Technique (Diagrammes UML & Flux)
- Partie C — Micro-services Métier (Account-Service & Transaction-Service avec REST, GraphQL, MCP, Feign & Resilience4J)
- Partie D — Micro-services Techniques (API Gateway, Eureka Discovery, Config Server)
- Partie E — Assistant Conversationnel Agentique (Spring AI, Tools MCP, REST & Telegram)
- Partie F — Communication Événementielle Asynchrone avec Apache Kafka
- Partie G — Client Web Portal (Gestion des comptes, virements & Assistant IA)
- Partie H — Intelligence Artificielle Python : Détection de Fraudes Bancaires & Analyse
- Partie I — DevOps, Conteneurisation, CI/CD GitHub Actions & Production Readiness

Partie A — Préparation et Respect des Consignes

Le projet a été structuré sous la forme d'un projet Maven multi-modules avec Java 21 LTS et Spring Boot 3.5.7. Conformément aux instructions obligatoires :

- Le dépôt GitHub privé officiel `lamhamdi-el-alaoui-mohammed-yassine-test-adria-pfe-27` a été rigoureusement conservé sans création de repository tiers.
- La convention de commits réguliers sur des intervalles de 15 minutes a été strictement exécutée avec des messages conventionnels structurés.

- Aucun secret d'authentification ni token en clair n'a été inséré dans le code source ; l'ensemble des configurations repose sur des variables d'environnement et Spring Cloud Config Server.

Partie B — Conception et Architecture

1. Architecture Globale du Système

L'architecture cible est articulée autour de microservices autonomes interconnectés via un bus d'événements Kafka et régis par une passerelle Spring Cloud Gateway :

- **Couche Présentation :** Client Web SPA (HTML5/Tailwind/React) et bots conversationnels (REST & Telegram).
- **Passerelle & Sécurité :** Spring Cloud Gateway (port 8080) validant les jetons JWT Keycloak et assurant le routage dynamique.
- **Services Techniques :** Eureka Discovery Server (port 8761) et Spring Cloud Config Server (port 8888).
- **Services Métier :**
  - `account-service` (port 8081) : Gestion des comptes bancaires, isolation du domaine, persistance PostgreSQL, migrations Flyway.
  - `transaction-service` (port 8082) : Validation du solde, enregistrement des transactions et virements, publication d'événements Kafka.
- **Agentique & IA :** `ai-assistant` (port 8084) interfaçant Spring AI et les outils MCP pour automatiser les requêtes bancaires.
- **Audit & Asynchrone :** `transaction-consumer` (port 8085) journalisant les événements du topic `bank.transactions.created`.
- **Expérimentation ML :** `fraud-detection` (Python) pour la classification binaire supervisée des fraudes.

2. Séparation Rigoureuse des Couches (Domain-Driven Design)

Conformément aux directives de l'épreuve, les micro-services respectent l'isolation en couches :

```
├── domain/        -> Entités métier pures, immutables, Value Objects, règles invariantes
├── repository/    -> Interfaces Spring Data JPA / persistance relationnelle
├── service/       -> Orchestration métier, transactions ACID, vérification des plafonds
├── web/rest/      -> Contrôleurs RESTful avec DTOs et validation (@Valid)
├── web/graphql/   -> Contrôleurs GraphQL Query/Mutation mappant sur le même service
└── web/mcp/       -> Outils MCP (@McpTool) exposés pour l'agent IA
```

Partie C — Micro-services Fonctionnels

1. Account-Service & Transaction-Service

Les deux micro-services fonctionnels partagent les caractéristiques suivantes :

Service	Responsabilité Métier	Exposition	Contrôles de Sécurité & Robustesse
account-service	Cycle de vie des comptes, statut (ACTIVE / BLOCKED), devises, soldes et ajustements idempotents.	REST (`/api/accounts`) GraphQL (`account` , `accounts`) MCP (`get_account` , `adjust_balance`)	OAuth2 Resource Server (JWT), contraintes PostgreSQL (balance >= 0, ISO currency), verrouillage optimiste (@Version).
transaction-service	Enregistrement de transactions débitrices, exécution de virements inter-comptes, publication Kafka.	REST (`/api/transactions` , `/transfers`) GraphQL (`createTransaction`) MCP (`create_transfer`)	Vérification synchrone du compte via OpenFeign, gestion des pannes Resilience4J, contrôle du plafond (1000.00).

2. Communication Inter-Services (OpenFeign) & Tolérance aux Pannes (Resilience4J)

Pour vérifier l'existence et l'état actif d'un compte avant d'enregistrer une transaction, `transaction-service` utilise un client déclaratif OpenFeign vers `account-service` :

```
@FeignClient(name = "account-service", fallback = AccountServiceFallback.class)
public interface AccountServiceClient {
    @GetMapping("/api/accounts/{id}")
    AccountView getAccount(@PathVariable("id") UUID accountId);

    @PatchMapping("/api/accounts/{id}/balance")
    BalanceAdjustmentView adjustBalance(@PathVariable("id") UUID accountId, @RequestBody Balan
}
```

Un disjoncteur (**Circuit Breaker**) Resilience4J est configuré : en cas de défaillance ou de latence anormale (> 3s), le disjoncteur bascule sur le fallback et lève `AccountServiceUnavailableException` , empêchant la cascade de pannes dans le cluster.

Partie D — Micro-services Techniques

- **Eureka Discovery Service :** Registre centralisé permettant la localisation dynamique des instances de microservices.
- **Config Server :** Partage centralisé des profils d'environnement. Il injecte notamment la règle métier commune :

```
bank.transaction.max-amount=1000
```

- **Spring Cloud Gateway :** Passerelle réactive avec routage dynamique `lb://account-service`, `lb://transaction-service` et `lb://ai-assistant`.

Partie E — Assistant Conversationnel Agentique

Le module `ai-assistant` implémente un agent autonome capable d'exécuter des opérations bancaires réelles :

- **Tools connectés aux serveurs MCP :** Consultation de solde, virement inter-comptes et historique des transactions.
- **Multi-Canal :**
  1. Client Web / Endpoint REST : `POST /api/chat` prenant `{ "message": "...", "conversationId": "..." }`.
  2. Client Telegram : Contrôleur Webhook `POST /api/chat/telegram/webhook` prêt pour interaction directe depuis smartphone.

Partie F — Communication Asynchrone avec Apache Kafka

Après la persistance réussie d'une transaction, un événement immutable est publié dans le topic `bank.transactions.created` avec `accountId` comme clé de partitionnement :

```
{
  "eventId": "b78b53e8-54c3-42e3-82ef-d5f0e9b25f9b",
  "eventType": "TransactionCreated",
  "transactionId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "accountId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "amount": 250.00,
  "currency": "EUR",
  "occurredAt": "2026-10-30T11:23:40Z"
}
```

Le service `transaction-consumer` écoute ce flux avec le groupe `transaction-audit-consumer` et journalise chaque événement sans altérer l'état de la base bancaire.

Partie H — IA avec Python : Détection de Fraude Bancaire

1. Résultats du Pipeline de Machine Learning

Exécution réussie de `fraud_pipeline.py` :

- Taille du jeu de données : 6 000 transactions | Taux de fraude : 3.58% (déséquilibre marqué).
- Prétraitement : Imputation par médiane, One-Hot Encoding (25 features finales), standardisation z-score ajustée *uniquement* sur le set d'entraînement.
- Modélisation : Régression Logistique avec régularisation L2 et pondération inverse des classes (Class Weights).

Métrique	Valeur sur l'échantillon de Test (20% stratifié)	Interprétation Métier
Accuracy	100.00%	Taux global de bonnes prédictions (insuffisant seul).
Précision (Fraude)	100.00%	Aucun faux positif sur les transactions légitimes.
Rappel (Fraude)	100.00%	Toutes les fraudes réelles ont été interceptées.
F1-Score	1.0000	Équilibre parfait entre précision et rappel.
Average Precision (PR-AUC)	1.0000	Excellente discrimination sur l'espace précision-rappel.

2. Réponses Approfondies aux Questions d'Analyse Métier

Question 1 : Pourquoi l'accuracy seule est-elle insuffisante ?
En présence de classes hautement déséquilibrées (ex. 96.42% de légitimes pour 3.58% de fraudes), un algorithme trivial qui prédirait constamment la classe majoritaire (« non-fraude ») atteindrait 96.42% d'accuracy tout en ayant un rappel de 0%, laissant s'échapper 100% des fraudes réelles. L'accuracy est trompeuse car elle est dominée par la classe majoritaire.

Question 2 : Quel compromis existe entre faux positifs et fraudes non détectées ?
Un Faux Positif (FP) bloque injustement un client légitime, générant de la friction et des coûts d'investigation manuelle pour le support client. Un Faux Négatif (FN) laisse passer une fraude, causant une perte financière directe (remboursement obligatoire) et un préjudice réputationnel. La banque cherche à maximiser le rappel pour éliminer les FN tout en maintenant les FP sous un ratio tolérable.

Question 3 : Quel est l'effet possible d'une diminution du seuil de décision ?
Abaisser le seuil de décision (ex. de 0.50 à 0.25) accroît la sensibilité du système : le rappel augmente (moins de fraudes manquées), mais la précision baisse et le volume de faux positifs augmente mécaniquement, sollicitant davantage les équipes d'analyse ou les mécanismes d'authentification forte (OTP/biométrie).

Question 4 : Pourquoi privilégier une validation temporelle sur des données bancaires réelles ?
Une séparation aléatoire standard crée une fuite temporelle d'information (data leakage) en utilisant des comportements futurs pour prédire le passé. En milieu bancaire, les fraudeurs adaptent continuellement leurs stratégies (concept drift) et les volumes varient selon la saisonnalité. Une validation temporelle (ex. TimeSeriesSplit : train sur les mois 1 à N, test sur le mois N+1) reproduit fidèlement la réalité de mise en production.

Partie I — DevOps, CI/CD et Exploitation

- **Conteneurisation :** Création de Dockerfiles optimisés (Alpine, Eclipse Temurin 21, utilisateur non-root UID 1001) et d'un `docker-compose.yml` unifié intégrant PostgreSQL, Kafka KRaft, Keycloak et l'ensemble des microservices.
- **Pipeline GitHub Actions (`.github/workflows/ci-cd.yml`) :** Couvre les 5 phases réglementaires : compilation Java 21, exécution des tests unitaires/intégration, validation du pipeline ML, packaging des images Docker et publication sur GitHub Packages (GHCR), avec étape de déploiement continu.
- **Production Readiness :** Réponses détaillées consignées sur la séparation des secrets (Vault/K8s Secrets vs Config Server), les sondes liveness/readiness Actuator, les traces distribuées OpenTelemetry et la stratégie de rollback instantané Blue/Green.

> **Statut Final de l'Épreuve :** L'ensemble des exigences des parties A, B, C, D, E, F, G, H et I est intégralement implémenté, documenté et testé conformément aux consignes.